

canlifal.com Flutter Uygulaması — Cursor AI Geliştirme Prompt'u

Bu prompt, canlifal.com (v1.0.131) Flutter uygulamasının web backend API'leri ile tam uyumlu çalışması için gerekli tüm bilgileri içerir.
Kopyalayıp Cursor AI'a yapıştırın.

TEMEL MİMARİ

Authentication (Dual Auth)

Backend tüm endpoint'lerde **dual auth** destekler:

- **Web**: NextAuth session (cookie-based)
- **Flutter/Mobil**: Bearer JWT token

Authorization: Bearer <accessToken>

Login endpoint: `POST /api/mobile/auth/login`

```
// Request:
{ "email": "user@email.com", "password": "..." }
// Response:
{ "accessToken": "...", "refreshToken": "...", "user": { "id", "name", "email", "role", "image" } }
```

Refresh endpoint: `POST /api/mobile/auth/refresh`

```
{ "refreshToken": "..." }
// Response:
{ "accessToken": "..." }
```

Token süreleri: access=7 gün, refresh=30 gün. NEXTAUTH_SECRET ile imzalanır.

CANLI YAYIN SİSTEMİ (Video Streams)

Base URL: `https://canlifal.com`

1. Yayın Listesi

```
GET /api/video-streams?page=1&limit=30
Auth: Opsiyonel
Response: {
  streams: [{ id, streamId, title, userId, status, likeCount, isLive, viewers, watching,
              broadcasterId, hostUserId, streamerName, thumbnailUrl, coverUrl, user:
{...} }],
  items: [...], // streams ile aynı
  pagination: { page, limit, total, totalPages }
}
```

2. Yayın Başlat

```
POST /api/video-streams
Auth: Bearer JWT (zorunlu, kullanıcı onaylı falci olmalı)
Body: { title, description?, category?, tags?, thumbnailUrl?, coverUrl? }
Response: { success: true, data: { id, streamId, title, ... } }
```

3. Yayın Detay

```
GET /api/video-streams/:streamId
Auth: Opsiyonel
Response: { id, streamId, title, userId, status, likeCount, isLive, viewers, watching,
            broadcasterId, hostUserId, streamerName, thumbnailUrl, coverUrl, ... }
```

4. Yayın Güncelle / Bitir

```
PATCH /api/video-streams/:streamId
Auth: Bearer JWT (yayın sahibi veya admin)
Body: { status?: 'ended', title?, description?, broadcastImage?, isImageMode?, backgroundUrl? }
```

5. Yayına Katıl (Viewer)

```
POST /api/video-streams/:streamId/join
Auth: Bearer JWT
Body: { nickname?, isHidden? }
Response: { viewerId, viewerCount }
```

6. Yayından Ayrıl

```
POST /api/video-streams/:streamId/leave
Auth: Bearer JWT
Body: { viewerId }
```

7. Beğeni (TikTok Tarzı — Her Tıklama +1)

```
POST /api/video-streams/:streamId/like
Auth: Opsiyonel (guest de beğenebilir)
Body: {} (boş)
Response: { likeCount: number }
```

ÖNEMLİ: Her POST çağrısı likeCount'u 1 artırır. Toggle değil, kümülatif.
Flutter'da hızlı tıklama destekleyin (debounce YAPMAYIN, her tap = 1 like).

8. Yorumlar

```
GET /api/video-streams/:streamId/comments
Response: [{ id, content, userId, user: { name, image, role, membership }, createdAt }]

POST /api/video-streams/:streamId/comments
Auth: Bearer JWT
Body: { content, nickname?, isHidden? }
Response: { id, content, userName, userImage, ... }
```

9. Hediye Gönder

```
GET /api/video-streams/gifts
Response: [{ id, name, icon, price, animation?, category }]

POST /api/video-streams/:streamId/gifts
Auth: Bearer JWT
Body: { giftTypeId, quantity?: 1 }
Response: { success, gift: {...}, newBalance, pkUpdate? }
```

10. Viewer Listesi

```
GET /api/video-streams/:streamId/viewers
Response: [{ userId, userName, userImage, joinedAt }]
```

11. Kamera Döndürme

Kamera döndürme tamamen **client-side** işlemdir. Backend'e sinyal gönderilmez.

```
// Flutter'da kamera switch:
await _cameraController.switchCamera(); // front ↔ back
// Veya:
final cameras = await availableCameras();
_currentCameraIndex = (_currentCameraIndex + 1) % cameras.length;
_cameraController = CameraController(cameras[_currentCameraIndex], ResolutionPre-
set.high);
await _cameraController.initialize();
```

Not: Web'deki gibi çalışır — sadece lokal kamera değişir, WebRTC peer connection otomatik güncellenir.

PK BATTLE SİSTEMİ

PK Durumu Sorgula

```
GET /api/video-streams/pk?streamId=xxx
Response: null | {
  id, stream1Id, stream2Id, user1Id, user2Id,
  score1, score2, status (pending|active|completed|rejected|cancelled),
  duration, startedAt, endedAt, winnerId,
  user1: { id, name, image }, user2: { id, name, image },
  stream1: { id, title }, stream2: { id, title }
}
```

PK Oluştur

```
POST /api/video-streams/pk
Auth: Bearer JWT (yayıncı)
Body: { action: 'create', streamId: 'benim-stream-id', targetStreamId: 'karşı-stream-id', duration?: 180 }
Response: { id, stream1Id, stream2Id, status: 'pending', ... }
```

PK Kabul Et

```
POST /api/video-streams/pk
Body: { action: 'accept', battleId: 'pk-id' }
Response: { id, status: 'active', startedAt, ... }
```

PK Reddet / İptal / Bitir

```
POST /api/video-streams/pk
Body: { action: 'reject' | 'cancel' | 'end', battleId: 'pk-id' }
```

PK Skor Güncelle

```
POST /api/video-streams/pk/score
Body: { battleId, streamId, points }
Response: { score1, score2 }
```

Aktif PK Listesi

```
GET /api/video-streams/pk/list
Response: [{ id, stream1Id, stream2Id, score1, score2, status, user1, user2, stream1, stream2, ... }]
```

PK Puanlama: Hediye gönderildiğinde otomatik güncellenir. Hediye toplam fiyatı PK skoruna eklenir.

SSE (Server-Sent Events) — Gerçek Zamanlı Güncellemeler

Video Stream SSE

```
GET /api/video-streams/:streamId/stream
Auth: Bearer JWT (header da)
Content-Type: text/event-stream
```

Events:

```
{ type: 'connected', streamId }
{ type: 'viewerCount', streamId, viewerCount }
{ type: 'streamMessage', streamId, comment: { id, content, userName, userImage, userId, role, membership } }
{ type: 'gift', streamId, gift: { giftName, giftIcon, senderName, senderImage, quantity, totalPrice, animation } }
{ type: 'streamEnded', streamId }
```

Chat Room SSE

```
GET /api/chat/rooms/:roomId/stream
Auth: Bearer JWT (header da)
```

Events:

```
{ type: 'connected', roomId }
{ type: 'messages', messages: [...] }
{ type: 'presence', users: [...] }
{ type: 'typing', users: [...] }
{ type: 'dj', event: 'QUEUE_UPDATED', playing, nowPlaying, musicUrl, musicQueue }
```

WebRTC Signaling

WebRTC sinyalleri DB üzerinden polling ile iletilir (WebSocket YOK).

Sinyal Gönder

```
POST /api/video-streams/signal
Auth: Bearer JWT
Body: {
  streamId,
  signalType: 'offer' | 'answer' | 'ice-candidate',
  receiverId,
  signalData: JSON.stringify({ sdp/candidate data })
}
```

Sinyal Al (Polling)

```
GET /api/video-streams/signal?streamId=xxx&recipientId=myUserId
Response: [{ id, type, senderId, data: { sdp/candidate }, createdAt }]
```

Polling aralığı: 1-2 saniye. Alınan sinyaller otomatik “processed” olarak işaretlenir.

Eski Sinyalleri Temizle (Yeniden Bağlantı İçin)

```
DELETE /api/room/signal?sessionId=xxx
Auth: Bearer JWT
```

Co-Broadcast (Birlikte Yayın)

Davet Gönder

```
POST /api/video-streams/:streamId/co-broadcast
Auth: Bearer JWT (yayın sahibi)
Body: { action: 'invite', userId: 'hedef-kullanıcı-id' }
```

Davet Kabul/Reddet

```
POST /api/video-streams/:streamId/co-broadcast
Body: { action: 'accept' | 'reject', coBroadcasterId: 'kayıt-id' }
```

Co-Broadcaster Listesi

```
GET /api/video-streams/:streamId/co-broadcast
Response: [{ id, userId, status, user: { id, name, image } }]
```

Maksimum 8 eşzamanlı co-broadcaster desteklenir.

MÜZİK / DJ SİSTEMİ (Sesli Oda)

Şarkı Ara (YouTube)

```
GET /api/music/search?q=şarkı+adı
Auth: Bearer JWT
Response: { items: [{ videoId, title, thumbnail, channelTitle, duration } ] }
```

Şarkı İsteği Gönder

```
POST /api/chat/rooms/:roomId/song-request
Auth: Bearer JWT
Body: {
  videoId: 'dQw4w9WgXcQ',
  title: 'Never Gonna Give You Up',
  duration?: '3:33',
  dedication?: 'Ali için',
  note?: 'Güzel şarkı',
  priority?: true,           // Sıra atlama (premium)
  skipPayment?: true        // Jeton düşmeden (admin/DJ)
}
Response: { success, newBalance, queued }
```

Kuyruk Sorgula

```
GET /api/chat/rooms/:roomId/song-request
Response: {
  queue: [{ id, videoId, title, dedication, note, duration, isPaid, userId, userName,
    createdAt }],
  playing: boolean,
  nowPlaying: { videoId, title, startedAt, duration } | null,
  musicUrl: 'https://youtube.com/watch?v=...' | null,
  musicQueue: [...queue]
}
```

Sıradaki Şarkıyı Çal

```
PATCH /api/chat/rooms/:roomId/song-request
Auth: Bearer JWT
Body: { requestId: 'mesaj-id' }
Response: { success }
```

Müzik Durumu

```
GET /api/chat/rooms/:roomId/music
Response: { videoId, title, startedAt, duration, playing, nowPlaying, musicUrl, music-
  Queue }
```

Müzik Ayarla / Durdur

```
POST /api/chat/rooms/:roomId/music
Auth: Bearer JWT (DJ yetkisi gerekli)
Body: { videoId, title, duration? }

DELETE /api/chat/rooms/:roomId/music
Auth: Bearer JWT (DJ yetkisi gerekli)
```

DJ Yönetimi

```
GET /api/chat/rooms/:roomId/dj
Response: {
  djUsers: [{ id, name, image, isPresent }],
  activeDjId, ownerPresent, canPlayMusic, isOwner,
  playing, nowPlaying, musicUrl, musicQueue
}

POST /api/chat/rooms/:roomId/dj
Auth: Bearer JWT (oda sahibi/admin)
Body: { action: 'add_dj' | 'remove_dj' | 'set_active_dj', userId: '...' }
```

SOHBET ODALARI

Oda Listesi

```
GET /api/chat/rooms?withCounts=true
Response: [{ id, slug, nameTr, nameEn, icon, onlineCount, recentUsers }]
```

Odaya Giriş (Presence)

```
POST /api/chat/rooms/:roomId/presence
Auth: Bearer JWT
Body: { nickname? }
```

Mesaj Gönder

```
POST /api/chat/rooms/:roomId/messages
Auth: Bearer JWT
Body: { content: 'Merhaba!' }
```

Mesaj Geçmişi

```
GET /api/chat/rooms/:roomId/messages?limit=50
Response: [{ id, content, userId, user: { name, role, nickname, chatRole,
roleSymbol }, createdAt }]
```

KULLANICI PROFİLİ

```
GET /api/user/profile
Auth: Bearer JWT
Response: { id, name, email, image, role, username, bio, jetonBalance, cfcBalance, ... }
```

```
PATCH /api/user/profile
Auth: Bearer JWT
Body: { name?, username?, bio?, image? }
```

Rumuz Değiştirme (Sohbet Odası İçin)

```
POST /api/chat/rooms/:roomId/presence
Auth: Bearer JWT
Body: { nickname: 'YeniRumuz' }
```

FAL İSTEME

Falcı Listesi

```
GET /api/fortune-tellers?sort=top Rated
Response: { tellers: [{ id, displayName, avatar, isOnline, rating, specialties, pricePerMinute }] }
```

Fal Tipleri

```
GET /api/homepage-fortune-cards
Response: [{ id, name, icon, image, href }]
```

Canlı Fal Oturumu Oluştur

```
POST /api/fortune-tellers/:tellerId/session
Auth: Bearer JWT
Body: { fortuneType: 'kahve_fali', maxMinutes: 10 }
Response: { sessionId, roomId, status }
```

CURSOR İÇİN GÖREV LİSTESİ

Aşağıdaki görevleri sırasıyla uygulayın:

1. Canlı Yayın — Kamera Döndürme

- `broadcast_screen.dart` dosyasında kamera switch butonu ekleyin
- `CameraController.switchCamera()` veya `availableCameras()` ile front/back geçişi
- WebRTC `RTCVideoRenderer` otomatik güncellenir, ek sinyal gerekmez

2. PK Battle Entegrasyonu

- PK düğmesi: Yayıncı aktif yayındayken diğer yayıncılara PK daveti gönderebilmeli
- `GET /api/video-streams/pk?streamId=xxx` ile aktif PK kontrol
- `POST /api/video-streams/pk` ile create/accept/reject/cancel/end
- PK aktifken split-screen UI gösterin (score1 vs score2)
- Hediye gönderildiğinde PK skoru otomatik güncellenir
- SSE'den gelen `gift` event'lerinde `pkUpdate` alanı kontrol edilmeli

3. Beğeni (TikTok Tarzı)

- Her dokunuşta `POST /api/video-streams/:streamId/like` çağırın
- Animasyonlu kalp efekti gösterin
- Debounce YAPMAYIN — her tap = +1 like

4. Fal İsteme

- `GET /api/fortune-tellers?sort=top Rated` ile falcı listesi
- Falcı profiline tıklayınca `POST /api/fortune-tellers/:tellerId/session` ile oturum oluşturun
- WebRTC ile canlı görüntülü görüşme başlat

5. Rumuz Değiştirme

- Sohbet odasında kullanıcı isminin yanında düzenle butonu
- POST /api/chat/rooms/:roomId/presence body: { nickname: 'YeniRumuz' }
- Değişiklik anında SSE üzerinden diğer kullanıcılara yansır

6. Müzik Sistemi

- GET /api/music/search?q=... ile YouTube arama
- POST /api/chat/rooms/:roomId/song-request ile şarkı isteği
- SSE { type: 'dj', event: 'QUEUE_UPDATED' } event'ini dinleyin
- YouTube player ile müzik çalın (youtube_player_flutter paketi)

7. SSE Bağlantısı

```
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<void> connectSSE(String streamId, String token) async {
  final client = http.Client();
  final request = http.Request('GET',
    Uri.parse('https://canlifal.com/api/video-streams/$streamId/stream'));
  request.headers['Authorization'] = 'Bearer $token';
  request.headers['Accept'] = 'text/event-stream';
  request.headers['Cache-Control'] = 'no-cache';

  final response = await client.send(request);
  response.stream
    .transform(utf8.decoder)
    .transform(const LineSplitter())
    .listen((line) {
      if (line.startsWith('data: ')) {
        final json = jsonDecode(line.substring(6));
        switch (json['type']) {
          case 'streamMessage':
            // Yorum geldi
            break;
          case 'gift':
            // Hediye geldi
            break;
          case 'viewerCount':
            // İzleyici sayısı güncellendi
            break;
          case 'streamEnded':
            // Yayın bitti
            break;
        }
      }
    });
}
```

8. WebRTC Signaling

```
// Sinyal gönder
await http.post(
  Uri.parse('https://canlifal.com/api/video-streams/signal'),
  headers: {'Authorization': 'Bearer $token', 'Content-Type': 'application/json'},
  body: jsonEncode({
    'streamId': streamId,
    'signalType': 'offer', // veya 'answer', 'ice-candidate'
    'receiverId': peerId,
    'signalData': jsonEncode(sdpData),
  }),
);

// Sinyal al (1-2 sn polling)
Timer.periodic(Duration(seconds: 1), (_) async {
  final res = await http.get(
    Uri.parse('https://canlifal.com/api/video-streams/signal?streamId=$streamId&recipientId=$myUserId'),
    headers: {'Authorization': 'Bearer $token'},
  );
  final signals = jsonDecode(res.body) as List;
  for (final signal in signals) {
    // signal['type'] = 'offer' | 'answer' | 'ice-candidate'
    // signal['data'] = SDP/ICE data
  }
});
```

ÖNEMLİ NOTLAR

1. **Base URL:** `https://canlifal.com` — tüm API çağrılarında kullanın
2. **Auth Header:** `Authorization: Bearer <accessToken>` — her authenticated çağrıda
3. **Content-Type:** `application/json` — POST/PATCH/DELETE isteklerinde
4. **Hata Formatı:** `{ error: 'Türkçe hata mesajı' }` — HTTP status code'a göre işleyin
5. **SSE:** EventSource yerine raw HTTP stream kullanın (Flutter'da EventSource paketi de kullanılabilir)
6. **WebRTC:** `flutter_webrtc` paketi önerilir. STUN server: `stun:stun.l.google.com:19302`
7. **Kamera:** `camera` paketi ile front/back switch. WebRTC `mediaStream.getVideoTracks()[0].switchCamera()` da çalışır.
8. **PK Score:** Hediye gönderildiğinde otomatik güncellenir — ekstra API çağrısı gerekmez
9. **Like:** Toggle DEĞİL, kümülatif. Her tap = +1.
10. **Rumuz:** `presence` endpoint'ine `nickname` parametresi gönderin